

หัวข้อโครงการ
StoreLink ระบบบริหารจัดการร้านค้า
รายชื่อผู้จัดทำ

1. 66012968 นางสาวสิริวรรณ จันแดง

2. 66080904 นางสาวกัณต์ณิชา อินตรา

นักศึกษาชั้นปีที่ 3 คณะเทคโนโลยีสารสนเทศ

สาขาวิทยาการคอมพิวเตอร์และนวัตกรรมการพัฒนาซอฟต์แวร์

1. บทนำ

1.1. ความเป็นมาและความสำคัญของปัญหา

ธุรกิจร้านค้าขนาดเล็กและขนาดกลางในปัจจุบันมีความหลากหลายมากขึ้น ไม่ว่าจะเป็นร้านค้าปลีก ร้านอาหาร หรือร้านบริการ เจ้าของกิจการส่วนใหญ่ต้องดูแลทั้งด้านการขาย การควบคุมสต็อกสินค้า การจัดการพนักงาน และการติดตามผลกำไรขาดทุน แต่กลับขาดเครื่องมือที่ช่วยบริหารจัดการอย่างเป็นระบบ ร้านค้าจำนวนมากยังพึ่งพาการจดบันทึกด้วยมือหรือโปรแกรมพื้นฐานทั่วไป ซึ่งก่อให้เกิดความผิดพลาดและไม่สามารถใช้ข้อมูลประกอบการตัดสินใจเชิงธุรกิจได้อย่างมีประสิทธิภาพ

ปัญหาหลักที่พบ ได้แก่

- ข้อมูลสินค้ากระจัดกระจาย ไม่สามารถตรวจสอบหรือออกรายงานได้แบบเรียลไทม์
- ขาดระบบบริหารพนักงานและสิทธิการเข้าถึงข้อมูล
- เจ้าของร้านไม่สามารถติดตามยอดขายและสต็อกได้จากระยะไกล
- ไม่มีระบบช่วยวิเคราะห์ยอดขายหรือสินค้าขายดีเพื่อวางแผนการตลาด

StoreLink จึงถูกพัฒนาขึ้นในฐานะ แพลตฟอร์มบริหารจัดการร้านค้าแบบครบวงจรที่เชื่อมโยงการขาย สต็อก พนักงาน และรายงานยอดขายไว้ในระบบเดียว โดยมีแนวคิดหลักคือ “ช่วยให้เจ้าของร้านเห็นภาพรวมธุรกิจได้จากที่เดียว และบริหารได้ง่ายขึ้น”

1.2. วัตถุประสงค์ของโครงการ

โครงการ StoreLink มีวัตถุประสงค์เพื่อพัฒนาแพลตฟอร์มบริหารจัดการร้านค้าแบบครบวงจร ที่ช่วยให้เจ้าของร้านและพนักงานสามารถจัดการงานภายในร้านได้อย่างเป็นระบบ สะดวก และมีประสิทธิภาพ โดยมีวัตถุประสงค์เชิงรายละเอียดดังต่อไปนี้

1. เพื่อพัฒนาแพลตฟอร์มบริหารจัดการร้านค้าแบบครบวงจรที่สามารถ ปรับใช้และรองรับ ได้กับหลากหลายประเภทธุรกิจ (เช่น ร้านค้าปลีก ร้านอาหาร)
2. เพื่อเพิ่มประสิทธิภาพในการจัดการข้อมูลสินค้าและสต็อกสินค้าคงเหลือ แบบเรียลไทม์ พร้อมระบบแจ้งเตือนที่ช่วยในการวางแผนการสั่งซื้อ
3. เพื่อพัฒนาระบบขายหน้าร้าน (POS) ที่ใช้งานง่าย รวดเร็ว และรองรับการทำธุรกรรม การลดราคา และการออกใบเสร็จอย่างเป็นระบบ
4. เพื่อพัฒนาระบบบริหารจัดการพนักงาน การกำหนดสิทธิ์การเข้าถึงข้อมูล (Role Management) และการตรวจสอบประวัติการทำงานเพื่อเพิ่มความปลอดภัยและความโปร่งใส
5. เพื่อพัฒนาระบบรายงานและการวิเคราะห์ข้อมูลทางธุรกิจที่สามารถ แสดงผลยอดขายและสถิติการดำเนินงาน ในรูปแบบที่ง่ายต่อการตัดสินใจเชิงกลยุทธ์
6. เพื่อสร้างระบบบริหารจัดการที่สามารถเข้าถึงและใช้งานได้ผ่าน Web Application จากทุกสถานที่ เพื่อให้เจ้าของร้านสามารถติดตามผลการดำเนินงานได้อย่างต่อเนื่อง
7. เพื่อออกแบบและพัฒนาระบบด้วยสถาปัตยกรรมที่ ยืดหยุ่นและขยายได้ (Scalable Architecture) เพื่อรองรับการต่อยอดและเพิ่มฟังก์ชันใหม่ในอนาคต

สรุปวัตถุประสงค์โดยรวม

โครงการ StoreLink มีวัตถุประสงค์เพื่อพัฒนาระบบบริหารจัดการร้านค้าแบบครบวงจร ที่สามารถรองรับร้านค้าหลายประเภทภายในระบบเดียว โดยมุ่งเน้นความสะดวกในการใช้งาน ความถูกต้องของข้อมูล การบริหารที่มีประสิทธิภาพ และความยืดหยุ่นในการขยายระบบ เพื่อช่วยยกระดับธุรกิจร้านค้าให้เข้าสู่การบริหารจัดการในยุคดิจิทัลอย่างสมบูรณ์

1.3. ขอบเขตของโครงการ

ขอบเขตของระบบ

StoreLink เป็นระบบบริหารจัดการร้านค้า (Store Management System) ที่ถูกออกแบบให้สามารถรองรับการใช้งานได้หลายประเภทธุรกิจ เช่น ร้านขายสินค้าทั่วไป ร้านอาหาร และร้านให้บริการ โดยระบบจะช่วยให้เจ้าของร้านสามารถจัดการข้อมูลสำคัญในร้านได้อย่างครบวงจร ทั้งการขายสินค้า การควบคุมสต็อก การจัดการพนักงาน และการดูรายงานยอดขาย

ประเภทร้านค้าที่ระบบรองรับ

ระบบ StoreLink ถูกออกแบบให้สามารถปรับใช้ได้กับร้านค้าหลายประเภท เพื่อให้ตอบโจทย์การดำเนินงานที่แตกต่างกันของแต่ละธุรกิจ ได้แก่

- ร้านค้าทั่วไป (Retail Store) — สำหรับร้านขายสินค้า เช่น ร้านของชำ ร้านเครื่องเขียน ร้านแฟชั่น หรือร้านอุปกรณ์ไอที รองรับการขายสินค้าแบบชิ้นและแบบล็อต รวมถึงสินค้าที่มี Serial Number
- ร้านอาหาร (Restaurant) — สำหรับร้านอาหารหรือคาเฟ่ รองรับระบบครัว (Kitchen Screen) และการจัดการเมนูอาหาร
- (ในอนาคต)ร้านให้บริการ (Service Shop) — สำหรับร้านตัดผม ร้านเสริมสวย ร้านล้างรถ หรือร้านซ่อมเครื่องใช้ไฟฟ้า รองรับการบริหารบันทึกบริการ การจัดทำตารางคิว และการชำระเงิน

ระบบมีคุณสมบัติสำคัญคือ รองรับการบริหารจัดการร้านค้าหลายแห่งภายใต้บัญชีเดียว (Multi-Store Management) ซึ่งช่วยให้เจ้าของร้านที่มีมากกว่าหนึ่งกิจการสามารถบริหารร้านทั้งหมดได้ในแพลตฟอร์มเดียว โดยข้อมูลของแต่ละร้านจะถูก จัดเก็บแยกออกจากกันอย่างชัดเจน (Data Isolation) เพื่อป้องกันความสับสนและความผิดพลาดของข้อมูล

รายละเอียดขอบเขตของระบบประกอบด้วย

- ระบบสามารถสร้างและจัดการร้านค้าได้หลายแห่งภายใต้บัญชีผู้ใช้เดียว โดยข้อมูลของแต่ละร้าน เช่น สินค้า พนักงาน การขาย และรายงาน จะถูกแยกออกจากกันอย่างอิสระ
- รองรับการขายสินค้าและบริการผ่านหน้าจอขายที่ใช้งานง่าย (Point-of-Sale Interface)
- สามารถเพิ่ม แก้ไข หรือลบข้อมูลสินค้าได้ พร้อมอัปเดตจำนวนสต็อกแบบเรียลไทม์
- รองรับการกำหนดราคาสินค้า ส่วนลด และภาษีได้อย่างยืดหยุ่น
- มีระบบจัดการพนักงาน สามารถเพิ่มบัญชีผู้ใช้งาน กำหนดตำแหน่งงาน และสิทธิ์การเข้าถึงข้อมูลได้
- มีระบบสรุปยอดขายและรายงานผลทางธุรกิจ เช่น ยอดขายรายวัน รายเดือน และรายการสินค้าหรือบริการที่ขายดี
- ระบบรองรับการใช้งานผ่านเว็บเบราว์เซอร์ สามารถเข้าถึงได้จากคอมพิวเตอร์หรือแท็บเล็ต

- ระบบต้องมี ความปลอดภัย โดยใช้ JWT สำหรับการยืนยันตัวตนและการเข้ารหัสที่สผ่าน และต้องมี ความเสถียร สามารถรองรับผู้ใช้งานพร้อมกัน (Concurrent Users) ได้

ระบบนี้ยังไม่รวมถึงฟังก์ชันการเชื่อมต่อกับระบบบัญชีภายนอก, การจัดการคลังสินค้าหลายสาขา (Warehouse System), และการจองสินค้ล่วงหน้า (Pre-Order System)

ขอบเขตของผู้ใช้งาน

ระบบ StoreLink แบ่งผู้ใช้งานออกเป็น 3 กลุ่มหลัก เพื่อให้สามารถเข้าถึงข้อมูลได้ตามหน้าที่และความรับผิดชอบของแต่ละบุคคล ดังนี้

1. ผู้ดูแลระบบ (Admin)

- มีสิทธิ์สูงสุดในการควบคุมระบบทั้งหมด
- ตรวจสอบข้อมูลร้านค้า ผู้ใช้งาน และสถานะการทำงานของระบบโดยรวม
- ดูแลความปลอดภัยและการบำรุงรักษาฐานข้อมูล

2. เจ้าของร้าน (Owner)

- สามารถสร้างและจัดการร้านค้าภายใต้บัญชีของตนเองได้หลายแห่ง
- เพิ่ม แก้ไข หรือลบข้อมูลสินค้าและบริการ
- จัดการพนักงานภายในร้าน และกำหนดสิทธิ์การเข้าถึงในแต่ละตำแหน่ง
- ตรวจสอบยอดขาย รายงาน และข้อมูลสรุปของแต่ละร้านได้แยกจากกัน
- สามารถใช้งานระบบได้ผ่านเว็บเบราว์เซอร์จากทุกที่

3. พนักงาน (Employee)

- เข้าสู่ระบบด้วยบัญชีผู้ใช้ของตนเอง
- ใช้งานฟังก์ชันการขายสินค้าและบริการภายในร้านที่ได้รับมอบหมาย
- ตรวจสอบข้อมูลสินค้า สต็อก และรายการขายตามสิทธิ์ที่ได้รับ
- ไม่สามารถเข้าถึงข้อมูลของร้านอื่นในบัญชีเดียวกันได้

1.4. เครื่องมือที่เกี่ยวข้องกับโครงการ

1.4.1. Hardware

System Model	ASUS TUF Dash F15 FX517ZC_FX517ZC
System Type	x64-based PC
Processor	12th Gen Intel(R) Core(TM) i5-12450H (2.00 GHz)
Installed RAM	16.0 GB
GPU	NVIDIA GeForce RTX 3050

1.4.2. Software

- React.js
ใช้สำหรับพัฒนาเว็บอินเทอร์เฟซ (Frontend) เพื่อสร้างหน้าจอผู้ใช้ที่สวยงาม ตอบสนองรวดเร็ว และรองรับการใช้งานบนหลายขนาดหน้าจอ
- Node.js + Express.js
ใช้สำหรับพัฒนา Web API ฝั่งเซิร์ฟเวอร์ (Backend) ทำหน้าที่เชื่อมต่อระหว่างผู้ใช้งานกับฐานข้อมูล และจัดการการร้องขอข้อมูลต่าง ๆ ภายในระบบ
- MongoDB
ใช้เป็นระบบฐานข้อมูลแบบ NoSQL สำหรับจัดเก็บข้อมูลร้านค้า สินค้า พนักงาน และยอดขาย โดยสามารถรองรับข้อมูลจากหลายร้านได้ในระบบเดียว
- JSON Web Token (JWT)
ใช้สำหรับยืนยันตัวตน (Authentication) และควบคุมสิทธิการเข้าถึงข้อมูล (Authorization) ของผู้ใช้งานแต่ละระดับ เพื่อเพิ่มความปลอดภัยของระบบ
- Git / GitHub
ใช้สำหรับจัดเก็บซอร์สโค้ด ควบคุมเวอร์ชันของโปรแกรม และทำงานร่วมกันภายในทีมพัฒนา
- Postman
ใช้ทดสอบและตรวจสอบการทำงานของ API รวมถึงการเชื่อมต่อระหว่างส่วน Frontend และ Backend

- Figma
ใช้ออกแบบโครงร่างหน้าจอ (Wireframe) และส่วนติดต่อผู้ใช้ (User Interface Design) เพื่อให้ระบบมีความสวยงามและใช้งานง่าย
- Visual Studio Code (VS Code)
ใช้เป็นเครื่องมือหลักในการพัฒนา เขียน และแก้ไขซอร์สโค้ดทั้งฝั่งเซิร์ฟเวอร์และฝั่งผู้ใช้

1.5. ตารางการดำเนินงาน

ขั้นตอนการดำเนินงาน	รายละเอียดการดำเนินงาน	ช่วงเวลาโดยประมาณ
ศึกษาปัญหาและความต้องการของระบบ	ศึกษาข้อมูลเกี่ยวกับระบบ POS ปัจจุบัน, วิเคราะห์ปัญหา, รวบรวมความต้องการของผู้ใช้	สัปดาห์ที่ 1
กำหนดขอบเขตและวัตถุประสงค์ของโครงการ	กำหนดขอบเขตระบบ, กลุ่มผู้ใช้เป้าหมาย, ฟังก์ชันที่ต้องการพัฒนา และบทบาทผู้ใช้งาน (Admin, Owner, Employee)	สัปดาห์ที่ 2
ออกแบบการทำงานของระบบ (System Design)	ออกแบบโครงสร้างฐานข้อมูล (Schema), ออกแบบหน้าจอ (Wireframe/UI), และกำหนด Flow การทำงานของระบบ	สัปดาห์ที่ 3-4
พัฒนาระบบ (System Development)	พัฒนา Backend (Node.js, Express, MongoDB) และ Frontend (HTML/CSS/JS), สร้าง API, หน้าจอ POS, และระบบจัดการพนักงาน	สัปดาห์ที่ 5-8
ทดสอบและปรับปรุงระบบ (Testing & Improvement)	ทดสอบการทำงานแต่ละส่วน เช่น การขายสินค้า, การบันทึกข้อมูล, การแสดงรายงาน และปรับปรุงข้อผิดพลาด	สัปดาห์ที่ 9
นำเสนอผลงาน (Presentation)	จัดทำเอกสารรายงาน, สรุปผลการพัฒนา, เตรียมสไลด์นำเสนอ และสาธิตระบบจริง	สัปดาห์ที่ 10

1.6. ประโยชน์ที่ความว่าจะได้รับ

- ช่วยให้เจ้าของร้านสามารถบริหารจัดการร้านค้าได้ในระบบเดียวอย่างมีประสิทธิภาพ ระบบ StoreLink รวมฟังก์ชันการทำงานที่สำคัญ เช่น การขายสินค้า การจัดการสต็อก การจัดการพนักงาน และรายงานยอดขายไว้ในแพลตฟอร์มเดียว ทำให้เจ้าของร้านสามารถควบคุมและติดตามกิจกรรมต่าง ๆ ได้จากจุดเดียวโดยไม่ต้องใช้หลายระบบแยกกัน
- ลดความซ้ำซ้อนและข้อผิดพลาดในการทำงาน ระบบช่วยลดการบันทึกข้อมูลด้วยมือ ซึ่งเป็นสาเหตุของข้อผิดพลาด เช่น การคำนวณยอดขายผิด หรือการนับสต็อกไม่ตรง ระบบจะอัปเดตข้อมูลแบบเรียลไทม์ ทำให้ข้อมูลถูกต้องและทันสมัยอยู่เสมอ
- เพิ่มความสะดวกและรวดเร็วในการขายสินค้าและบริการ พนักงานสามารถค้นหาและบันทึกการขายได้ง่าย ระบบจะคำนวณราคาสุทธิ ภาษี และส่วนลดโดยอัตโนมัติ ช่วยลดเวลาการทำงานและเพิ่มความพึงพอใจให้กับลูกค้า
- ช่วยให้เจ้าของร้านสามารถตรวจสอบยอดขายและข้อมูลทางธุรกิจได้แบบเรียลไทม์ ระบบรายงานของ StoreLink แสดงยอดขาย สินค้าขายดี และสถิติการดำเนินงานในรูปแบบกราฟและตาราง เจ้าของร้านสามารถใช้ข้อมูลเหล่านี้ในการวางแผนธุรกิจ การจัดการสต็อก และการประเมินประสิทธิภาพของพนักงานได้อย่างแม่นยำ
- เพิ่มความปลอดภัยและความเป็นระบบในการจัดการพนักงาน ระบบมีการกำหนดสิทธิ์ผู้ใช้งาน (Role Management) แยกตามตำแหน่งงาน เช่น เจ้าของร้าน ผู้จัดการ และพนักงานขาย เพื่อป้องกันการเข้าถึงข้อมูลที่ไม่เกี่ยวข้องและเพิ่มความโปร่งใสในการทำงานภายในร้าน
- รองรับการบริหารร้านค้าหลายแห่งในบัญชีเดียว (Multi-Store Management) เจ้าของร้านสามารถบริหารจัดการร้านค้าหลายแห่งได้ภายในระบบเดียว โดยข้อมูลของแต่ละร้าน เช่น สินค้า พนักงาน และยอดขาย จะถูกแยกเก็บอย่างอิสระ ทำให้สามารถติดตามผลการดำเนินงานของแต่ละร้านได้ชัดเจนและปลอดภัย
- ช่วยลดต้นทุนด้านการจัดการและเพิ่มประสิทธิภาพการดำเนินงานของธุรกิจ ระบบช่วยลดภาระในการจัดทำเอกสาร ลดเวลาการจัดการข้อมูลซ้ำซ้อน และช่วยให้พนักงานทำงานได้รวดเร็วขึ้น ส่งผลให้ธุรกิจมีประสิทธิภาพและลดค่าใช้จ่าย

ในระยะยาว

- สามารถเข้าถึงระบบได้จากทุกที่ทุกเวลา
เนื่องจาก StoreLink ทำงานผ่านเว็บแอปพลิเคชัน เจ้าของร้านสามารถเข้าถึงระบบผ่านอินเทอร์เน็ตได้ตลอดเวลา ไม่ว่าจะอยู่ที่ร้านหรืออยู่นอกสถานที่ ช่วยเพิ่มความยืดหยุ่นในการบริหารงาน
- ส่งเสริมให้ร้านค้าขนาดเล็กและขนาดกลางก้าวสู่การบริหารในยุคดิจิทัล
ระบบ StoreLink ช่วยให้ร้านค้าธรรมดาสามารถใช้เทคโนโลยีจัดการธุรกิจได้อย่างมืออาชีพ โดยไม่ต้องลงทุนสูง ทำให้สามารถแข่งขันในตลาดและขยายกิจการได้อย่างยั่งยืน

2. ทฤษฎีการวิเคราะห์การออกแบบระบบและงานวิจัยที่เกี่ยวข้อง

2.1. การวิเคราะห์และออกแบบระบบ

2.1.1. การวิเคราะห์และออกแบบระบบ (System Analysis and Design)

การวิเคราะห์และออกแบบระบบ (System Analysis and Design) เป็นขั้นตอนสำคัญในการพัฒนาระบบสารสนเทศ เพื่อให้ระบบสามารถตอบสนองต่อความต้องการของผู้ใช้งานได้อย่างถูกต้องและมีประสิทธิภาพ การวิเคราะห์ระบบจะเริ่มจากการศึกษาปัญหาของระบบเดิม การเก็บรวบรวมข้อมูลจากผู้ใช้งาน และการกำหนดความต้องการของระบบใหม่ (System Requirement) จากนั้นจึงนำข้อมูลเหล่านั้นมาวางแผนการออกแบบระบบให้สามารถแก้ปัญหาได้ตรงจุด

ในการพัฒนา ระบบ StoreLink มีการประยุกต์ใช้แนวคิดและกระบวนการออกแบบระบบเชิงวัตถุ (Object-Oriented Analysis and Design: OOAD) เพื่อให้แต่ละส่วนของระบบสามารถทำงานแยกจากกันได้อย่างอิสระและสะดวกต่อการบำรุงรักษา ระบบถูกออกแบบให้มีโครงสร้างแบบ Client-Server Architecture ซึ่งแบ่งออกเป็นสองส่วนหลัก ได้แก่

1. ส่วนผู้ใช้งาน (Client Side / Frontend)

ใช้เทคโนโลยี React.js เพื่อพัฒนาอินเทอร์เฟซที่ตอบสนองรวดเร็ว ใช้งานง่าย และเหมาะกับร้านค้าหลายประเภท เช่น ร้านขายสินค้า ร้านอาหาร หรือร้านให้บริการ

2. ส่วนเซิร์ฟเวอร์ (Server Side / Backend)

ใช้ Node.js และ Express.js ในการพัฒนา Web API เพื่อเชื่อมต่อข้อมูลระหว่างผู้ใช้กับฐานข้อมูล รวมถึงจัดการกระบวนการทางธุรกิจ เช่น การขายสินค้า การจัดการสต็อก และ

การสร้างรายงานยอดขาย

3. ส่วนฐานข้อมูล (Database)

ใช้ MongoDB ซึ่งเป็นฐานข้อมูลแบบ NoSQL ที่สามารถเก็บข้อมูลได้ยืดหยุ่น เหมาะกับระบบที่ต้องรองรับร้านค้าหลายแห่งในบัญชีเดียว โดยแต่ละร้านจะมีข้อมูลแยกจากกัน อย่างอิสระ (Data Isolation) เพื่อป้องกันความซ้ำซ้อนและเพิ่มความปลอดภัยของข้อมูล

4. การออกแบบสถาปัตยกรรมระบบ (System Architecture Design)

ระบบ StoreLink ใช้แนวคิดการออกแบบแบบ Modular Design แบ่งการทำงานออกเป็นโมดูลย่อย เช่น โมดูลจัดการสินค้า โมดูลพนักงาน โมดูลรายงาน และโมดูลร้านค้า เพื่อให้สามารถปรับปรุงและขยายระบบได้ง่ายในอนาคต

5. การออกแบบส่วนติดต่อผู้ใช้ (User Interface Design)

ระบบได้รับการออกแบบด้วยเครื่องมือ Figma เพื่อให้ส่วนติดต่อผู้ใช้มีความเป็นมิตร ใช้งานสะดวก และเหมาะสมกับลักษณะงานของผู้ใช้งานแต่ละบทบาท เช่น เจ้าของร้าน พนักงาน และผู้ดูแลระบบ

6. การวิเคราะห์การทำงานของระบบ (System Workflow Analysis)

ในขั้นตอนการวิเคราะห์ระบบ ได้มีการสร้าง Use Case Diagram เพื่อแสดงความสัมพันธ์ระหว่างผู้ใช้งานกับระบบ และ Data Flow Diagram (DFD) เพื่ออธิบายการไหลของข้อมูลในแต่ละกระบวนการ เช่น การขายสินค้า การจัดการสต็อก และการสร้างรายงานยอดขาย

กล่าวโดยสรุป การวิเคราะห์และออกแบบระบบของ StoreLink ได้เน้นหลักการความถูกต้อง ความยืดหยุ่น และความสามารถในการขยายระบบในอนาคต โดยยึดตามหลักการออกแบบระบบเชิงวัตถุและเทคโนโลยีเว็บสมัยใหม่ เพื่อให้สามารถตอบโจทย์ธุรกิจร้านค้าหลากหลายประเภทได้อย่างมีประสิทธิภาพ

2.2. งานวิจัยที่เกี่ยวข้อง

จากการศึกษางานวิจัยและระบบที่มีแนวคิดคล้ายคลึงกับ StoreLink พบว่าระบบบริหารจัดการร้านค้าและระบบขายหน้าร้าน (Point of Sale System) ได้ถูกพัฒนามาอย่างต่อเนื่อง เพื่อช่วยเพิ่มประสิทธิภาพในการดำเนินงานของธุรกิจขนาดเล็กและขนาดกลาง โดยมีงานที่เกี่ยวข้องดังนี้

1. งานวิจัยเรื่อง “การพัฒนาระบบบริหารจัดการร้านค้าปลีกด้วยเทคโนโลยีเว็บแอปพลิเคชัน” (2565)

งานวิจัยนี้ได้พัฒนาระบบขายหน้าร้านสำหรับร้านค้าปลีกขนาดเล็ก โดยมีฟังก์ชันการขายสินค้า การจัดการสต็อก และการออกรายงานยอดขาย ซึ่งมีแนวคิดคล้ายกับ StoreLink ในด้านการรวมฟังก์ชันทั้งหมดไว้ในระบบเดียว แต่ StoreLink ได้พัฒนาเพิ่มเติมโดยขยายให้รองรับร้านอาหารและร้านบริการได้ด้วย

2. งานวิจัยเรื่อง “ระบบบริหารจัดการร้านอาหารออนไลน์” (2566)

งานวิจัยนี้พัฒนาระบบสำหรับร้านอาหารที่สามารถรับออเดอร์จากลูกค้าและส่งข้อมูลไปยังครัวได้โดยอัตโนมัติ มีการจัดการโต๊ะอาหารและการออกใบเสร็จอิเล็กทรอนิกส์ แนวคิดนี้ถูกนำมาประยุกต์ใช้ใน StoreLink โดยขยายให้ระบบสามารถเลือกโหมดร้านอาหารได้ภายในแพลตฟอร์มเดียวกัน

3. งานวิจัยเรื่อง “ระบบจองคิวออนไลน์สำหรับร้านบริการ” (2564)

งานวิจัยนี้เสนอระบบจองคิวที่ช่วยให้ลูกค้าสามารถนัดหมายการใช้บริการผ่านเว็บไซต์ได้ ลดปัญหาความแออัดและการจัดการคิวผิดพลาด ซึ่งแนวคิดนี้ได้นำมาประยุกต์ใช้กับโมดูลร้านให้บริการใน StoreLink เพื่อให้เจ้าของร้านสามารถจัดตารางงานและบริหารพนักงานได้อย่างเป็นระบบ

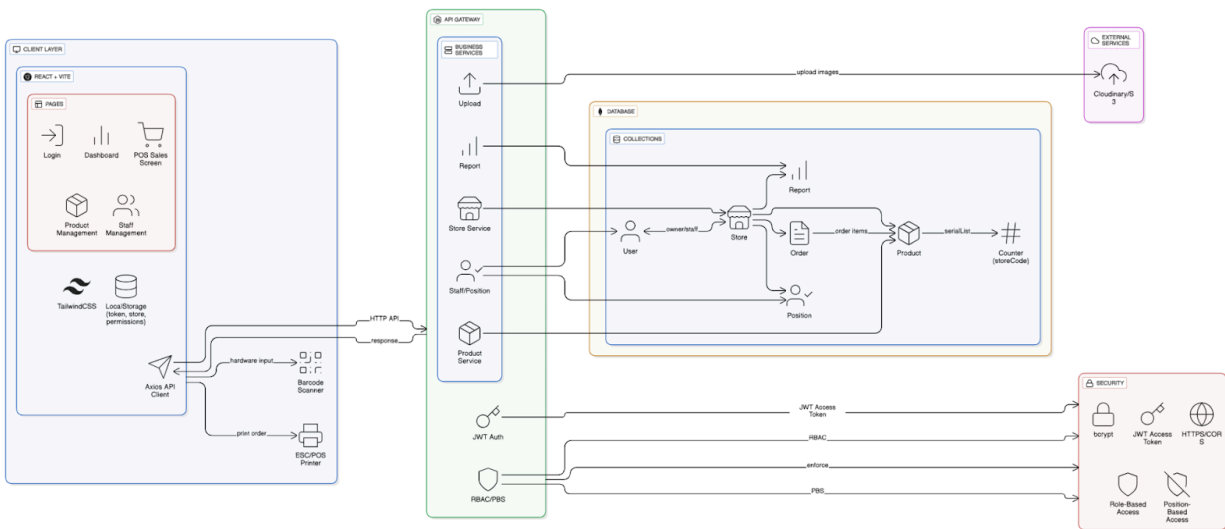
4. ระบบขายหน้าร้านของร้านสะดวกซื้อ 7-Eleven (กรณีศึกษาเชิงระบบจริง)

จากการศึกษาระบบของร้าน 7-Eleven พบว่าใช้เทคโนโลยี POS ร่วมกับระบบคลังสินค้าแบบเรียลไทม์ สามารถติดตามยอดขายและสต็อกสินค้าได้อย่างถูกต้อง ซึ่งแนวทางดังกล่าวถูกนำมาปรับใช้กับ StoreLink โดยใช้แนวคิดการอัปเดตข้อมูลแบบเรียลไทม์ (Real-Time Update) และการแยกข้อมูลตามร้าน (Data Isolation) เพื่อให้เหมาะสมกับธุรกิจขนาดเล็กถึงกลาง

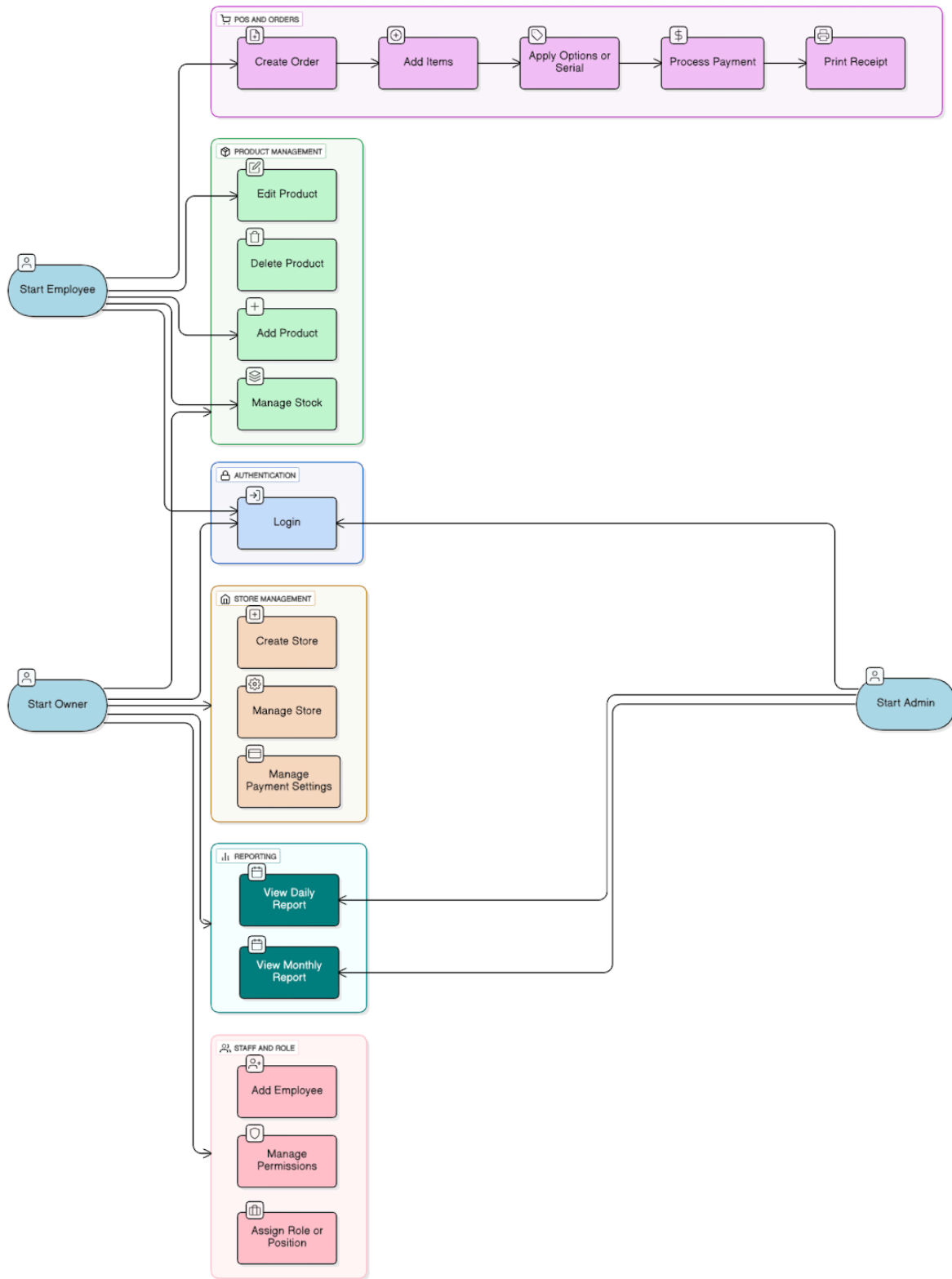
จากการศึกษาและเปรียบเทียบระบบที่เกี่ยวข้อง พบว่า StoreLink มีจุดเด่นคือการเป็นแพลตฟอร์มบริหารจัดการร้านค้าหลายประเภทในระบบเดียว รองรับร้านค้าได้หลายแห่งในบัญชีเดียว และมีฟังก์ชันที่ครอบคลุมตั้งแต่การขายสินค้า การจัดการสต็อก การบริหารพนักงาน ไปจนถึงการสรุปผลยอดขาย จึงสามารถนำแนวคิดจากงานวิจัยที่มีอยู่มาปรับใช้ให้เหมาะสมกับบริบทธุรกิจยุคปัจจุบันได้อย่างมีประสิทธิภาพ

3. วิธีการดำเนินโครงการ

3.1. System Architecture



3.2. Case Diagram



3.3. Data Schema

3.3.1. User

```
const userSchema = new mongoose.Schema({
  firstName: { type: String, required: true },
  lastName: { type: String, required: true },
  gender: { type: String, enum: ["Male", "Female", "Other"], default: null },
},
  birthDate: { type: Date, default: null },
  idCard: { type: String, default: null },
  idCardImage: { type: String, default: null },
  phone: { type: String, default: null },
  email: { type: String, default: null },
  address: { type: String, default: null },
  emergencyContact: { type: String, default: null },
  username: { type: String, unique: true, sparse: true },
  password: { type: String, required: true },
  role: { type: String, enum: ["Admin", "Owner", "Employee"], default: "Owner" },
  storeIds: [{ type: mongoose.Schema.Types.ObjectId, ref: "Store" }],
  positionId: { type: mongoose.Schema.Types.ObjectId, ref: "Position" },
  status: { type: String, enum: ["active", "inactive"], default: "active" },
},
  hireDate: { type: Date, default: Date.now },
  resignDate: { type: Date, default: null },
  salary: { type: Number, default: null },
  note: { type: String, default: null },
}, { timestamps: true });
```

3.3.2. Position

```
const positionSchema = new mongoose.Schema({
  storeId: { type: mongoose.Schema.Types.ObjectId, ref: "Store", required: true },
  positionName: { type: String, required: true },
  permissions: [{ type: String }],
}, { timestamps: true });
```

3.3.3. Store

```
const storeSchema = new mongoose.Schema({
  {
    storeCode: { type: String, unique: true },
    ownerId: { type: mongoose.Schema.Types.ObjectId, ref: "User",
required: true },
    storeName: { type: String, required: true },
    storeImage: { type: String, default: "" },
    storeType: {
      type: String,
      enum: ["general", "restaurant", "service"],
      required: true,
    },
  },
  address: String,
  phone: String,
  taxRate: { type: Number, default: 0 },
  paymentSettings: { type: paymentSettingsSchema, default: () => ({} ) },
},
{ timestamps: true }
);
```

3.3.4. Product

```
const productSchema = new mongoose.Schema({
  storeId: { type: mongoose.Schema.Types.ObjectId, ref: "Store",
required: true },
  barcode: { type: String, unique: true, sparse: true },
  name: { type: String, required: true },
  category: String,
  type: { type: String, enum: ["standard", "serialized"], default:
"standard" },
  price: { type: Number, required: true },
  unit: String,
```

```

duration: { type: Number, default: 0 },
stockQty: { type: Number, default: 0 },
optionGroups: [
  {
    name: { type: String, required: true }, // e.g., "Sweetness",
    "Toppings"
    selectionType: { type: String, enum: ["single", "multiple"],
default: "single" },
    required: { type: Boolean, default: false },
    maxSelections: { type: Number },
    choices: [
      {
        label: { type: String, required: true },
        value: { type: String },
        priceDelta: { type: Number, default: 0 },
        isDefault: { type: Boolean, default: false }
      }
    ]
  }
],
serialList: [
  {
    serialNumber: { type: String, unique: true, sparse: true },
    status: { type: String, enum: ["available", "sold"], default:
"available" }
  }
],
status: { type: String, enum: ["available", "unavailable"], default:
"available" },
description: String,
productImage: { type: String, default: "" },
}, { timestamps: true });

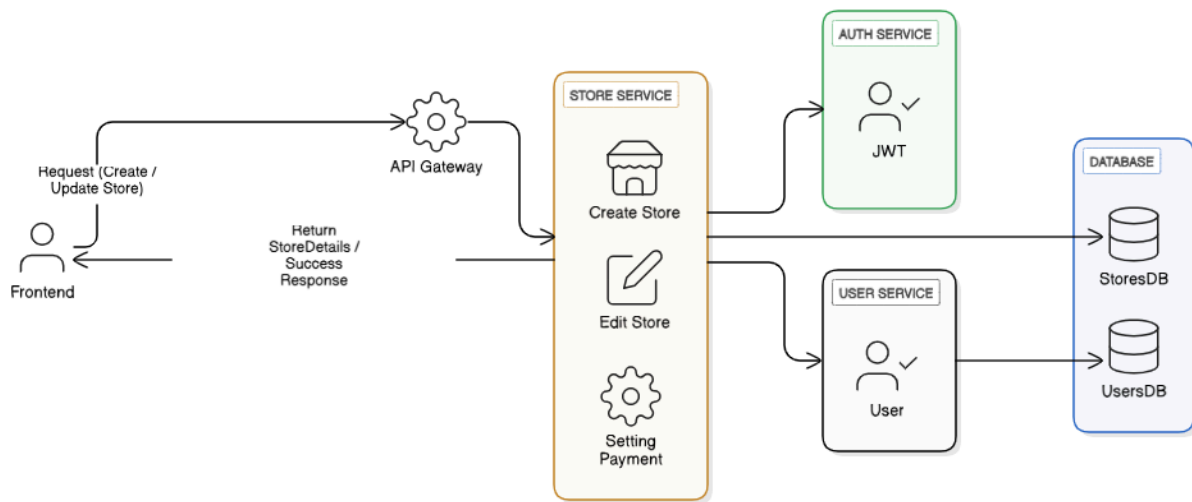
```

3.3.5. Order

```
const orderSchema = new mongoose.Schema({
  storeId: { type: mongoose.Schema.Types.ObjectId, ref: "Store",
required: true },
  userId: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
  kitchenStatus: {
    type: String,
    enum: ["waiting", "cooking", "done"],
    default: "waiting"
  },
  items: [orderItemSchema],
  subTotal: Number,
  tax: Number,
  total: Number,
  paymentMethod: {
    type: String,
    enum: ["cash", "promptpay", "other", "none"],
    default: "none",
  },
  paidAmount: { type: Number, default: 0 },
  changeAmount: { type: Number, default: 0 },
  status: {
    type: String,
    enum: ["pending", "paid", "cancelled"],
    default: "pending",
  },
  receiptNumber: { type: String },
  paidAt: { type: Date },
  createdAt: { type: Date, default: Date.now },
  updatedAt: { type: Date, default: Date.now },
});
```

<https://www.mermaidchart.com/d/1966c13d-da72-4978-a47e-ab3bf012eeda>

3.4. Report Microservices Architecture Design



eraser

Admin

จัดการข้อมูลร้านค้าสำหรับผู้ดูแลระบบ

GET /admin/stores

ดึงข้อมูลร้านค้าทั้งหมด

Parameters

No parameters

Responses

Code	Description	Links
200	✓ ดึงข้อมูลร้านค้าทั้งหมดสำเร็จ Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": "672c7d482dbf4d15462f9c25", "storeName": "ร้านครัวพลอย", "storeType": "restaurant", "ownerId": { "name": "พลอย สิริวรรณ", "email": "ploy@gmail.com" } }</pre>	No links
500	✗ เกิดข้อผิดพลาดในการดึงข้อมูลร้านค้า	No links

เปิดใช้งาน

DELETE

/admin/stores/{id}

ลบร้านค้าออกจากระบบ

Parameters

Try it out

Name	Description
id * required string (path)	รหัสร้านค้า (Store ID) 672c7d482dbf4d15d62f9c25

Responses

Code	Description	Links
200	✔️ ลบร้านค้าสำเร็จ	No links
404	❌ ไม่พบร้านค้าที่ต้องการลบ	No links
500	❌ เกิดข้อผิดพลาดในระบบ	No links

Auth

ระบบจัดการการเข้าสู่ระบบและบัญชีผู้ใช้

POST

/auth/register

สมัครสมาชิกใหม่

Parameters

Try it out

No parameters

Request body required

application/json

Example Value

Schema

```
{
  "name": "พลอย สิวารรรถ",
  "email": "ploy@gmail.com",
  "password": "123456"
}
```

Responses

Code	Description	Links
201	✔️ สมัครสมาชิกสำเร็จ	No links
400	❌ ข้อมูลไม่ถูกต้อง	No links

POST

/auth/login

เข้าสู่ระบบ

Parameters

Try it out

No parameters

Request body required

application/json

Example Value

Schema

```
{
  "email": "ploy@gmail.com",
  "password": "123456"
}
```

Responses

Code	Description	Links
200	✔️ เข้าสู่ระบบสำเร็จและส่ง JWT token กลับ	No links
401	❌ Email หรือรหัสผ่านไม่ถูกต้อง	No links

GET

/auth/profile

ดึงข้อมูลโปรไฟล์ผู้ใช้ปัจจุบัน

🔒 ⬆

Parameters

Try it out

No parameters

Responses

Code	Description	Links
200	✅ ดึงข้อมูลสำเร็จ	No links
401	❌ ไม่พบ token หรือ token ไม่ถูกต้อง	No links

Employeesจัดการข้อมูลพนักงานในร้านค้า

GET

/employees/{storeId}

ดึงข้อมูลพนักงานทั้งหมดในร้าน

🔒 ⬆

Parameters

Try it out

Name	Description
storeId required	รหัสร้านค้า
string (path)	storeId

Responses

Code	Description	Links
200	✅ ดึงข้อมูลพนักงานสำเร็จ	No links
Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": "string", "firstName": "พณณ", "lastName": "ธัญวณณ", "email": "plop@example.com", "position": "แอดมินเอร์"} }</pre>		
403	❌ ไม่มีสิทธิ์เข้าถึง	No links
500	❌ เซิร์ฟเวอร์ผิดพลาด	No links

เปิดใช้งาน
โปรไฟล์

POST

/employees/create/{storeId} เพิ่มพนักงานใหม่ในร้าน

Parameters

Try it out

Name	Description
storeId * required	
string	storeId
(path)	

Request body required

application/json

Example Value | Schema

```
{  "firstName": "พลาบ",  "lastName": "สีจิวสี",  "email": "pjoy@sample.com",  "password": "123456",  "positionId": "674fe2a5a64f452fd9bd4291"}  
```

Responses

Code	Description	Links
201	✓ เพิ่มพนักงานสำเร็จ	No links
400	✗ ข้อมูลไม่ถูกต้อง	No links
403	✗ ไม่มีสิทธิ์เข้าถึง	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

PUT

/employees/update/{id} แก้ไขข้อมูลพนักงาน

Parameters

Try it out

Name	Description
id * required	รหัสพนักงาน
string	id
(path)	

Request body required

application/json

Example Value | Schema

```
{  "firstName": "string",  "lastName": "string",  "positionId": "string",  "email": "string"}  
```

Responses

Code	Description	Links
200	✓ อัปเดตข้อมูลพนักงานสำเร็จ	No links
404	✗ ไม่พบพนักงาน	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

DELETE

/employees/delete/{id}

ลบพนักงานออกจากระบบ

Parameters

Try it out

Name	Description
id * required	รหัสพนักงาน
string (path)	id

Responses

Code	Description	Links
200	✓ ลบพนักงานสำเร็จ	No links
404	✗ ไม่พบพนักงาน	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

Orders

จัดการคำสั่งซื้อ ใบเสร็จ และสถานะห้องครัว

POST

/orders/create

สร้างออเดอร์ใหม่ (เบ็ดเตล็ด)

Parameters

Try it out

No parameters

Request body required

application/json

Example Value | Schema

```
{  "storeId": "string",  "userId": "string",  "items": [    {      "productId": "string",      "name": "string",      "qty": 0,      "price": 0    }  ],  "subTotal": 0,  "tax": 0,  "total": 0,  "paymentMethod": "cash",  "isInstantPay": false}
```

Responses

Code	Description	Links
201	✓ สร้างออเดอร์สำเร็จ	No links
400	✗ ข้อมูลไม่ถูกต้อง	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

มีลิงก์จาก
ไปที่ การส่ง

PUT

/orders/{orderId}/add-item

เพิ่มสินค้าเข้าใบสั่งซื้อ(ยังไม่ทำ)

Parameters

Try it out

Name	Description
orderId * required	รหัสออเดอร์
string (path)	orderId

Request body required

application/json

Example Value | Schema

```
{  "productId": "string",  "qty": 0}
```

Responses

Code	Description	Links
200	✅ เพิ่มสินค้าใบสั่งซื้อสำเร็จ	No links
404	❌ ไม่พบออเดอร์	No links
500	❌ เซิร์ฟเวอร์ผิดพลาด	No links

PUT

/orders/{orderId}/pay

ชำระเงินสำหรับออเดอร์

Parameters

Try it out

Name	Description
orderId * required	รหัสออเดอร์
string (path)	orderId

Request body required

application/json

Example Value | Schema

```
{  "paymentMethod": "string",  "paidAmount": 0,  "changeAmount": 0}
```

Responses

Code	Description	Links
200	✅ ชำระเงินสำเร็จ	No links
400	❌ ข้อมูลไม่ถูกต้อง	No links
404	❌ ไม่พบออเดอร์	No links

GET

/orders/{orderId}/receipt

ดึงข้อมูลใบเสร็จของออเดอร์

Parameters

Try it out

Name	Description
orderId * <i>required</i>	รหัสออเดอร์
string (path)	orderId

Responses

Code	Description	Links
200	✅ ส่งใบเสร็จสำเร็จ	No links
404	❌ ไม่พบออเดอร์	No links
500	❌ เซิร์ฟเวอร์ผิดพลาด	No links

PUT

/orders/{orderId}/cancel

ยกเลิกออเดอร์(ยังไม่ทำ)

Parameters

Try it out

Name	Description
orderId * <i>required</i>	รหัสออเดอร์
string (path)	orderId

Responses

Code	Description	Links
200	✅ ยกเลิกออเดอร์สำเร็จ	No links
404	❌ ไม่พบออเดอร์	No links
500	❌ เซิร์ฟเวอร์ผิดพลาด	No links

GET

/orders/{storeId}/all-receipts

ดึงใบเสร็จทั้งหมดของร้านค้า

Parameters

Try it out

Name	Description
storeId * <i>required</i>	รหัสร้านค้า
string (path)	storeId

Responses

Code	Description	Links
200	✅ ส่งข้อมูลใบเสร็จสำเร็จ	No links
500	❌ เซิร์ฟเวอร์ผิดพลาด	No links

PUT

/orders/{orderId}/kitchen-status

อัปเดตสถานะของออเดอร์ในห้องครัว

Parameters

Try it out

Name	Description
orderId * required	รหัสออเดอร์
string (path)	orderId

Request body required

application/json

Example Value

Schema

```
{
  "kitchenStatus": "waiting"
}
```

Responses

Code	Description	Links
200	✓ อัปเดตสถานะครัวสำเร็จ	No links
400	✗ ข้อมูลไม่ถูกต้อง	No links
404	✗ ไม่พบออเดอร์	No links

Positions

จัดการตำแหน่งและสิทธิ์ของพนักงานในร้านค้า

GET

/positions/{storeId}

ดึงข้อมูลตำแหน่งทั้งหมดในร้าน

Parameters

Try it out

Name	Description
storeId * required	รหัสร้านค้า
string (path)	storeId

Responses

Code	Description	Links
200	✓ ดึงข้อมูลตำแหน่งสำเร็จ	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

POST

/positions/create/{storeId}

สร้างตำแหน่งใหม่ในร้าน

Parameters

Try it out

Name	Description
storeId * required	รหัสร้านค้า
string (path)	storeId

Request body required

application/json

Example Value

Schema

```
{
  "name": "แคชเชียร์",
  "permissions": [
    "sale",
    "all_receipts"
  ]
}
```

Responses

Code	Description	Links
201	เพิ่มตำแหน่งสำเร็จ	No links
400	ข้อมูลไม่ถูกต้อง	No links
500	เซิร์ฟเวอร์ผิดพลาด	No links

PUT

/positions/update/{id}

แก้ไขข้อมูลตำแหน่ง

Parameters

Try it out

Name	Description
id * required	รหัสตำแหน่ง
string (path)	id

Request body required

application/json

Example Value

Schema

```
{
  "name": "string",
  "permissions": [
    "string"
  ]
}
```

Responses

Code	Description	Links
200	อัปเดตข้อมูลสำเร็จ	No links
404	ไม่พบตำแหน่ง	No links
500	เซิร์ฟเวอร์ผิดพลาด	No links

เปิดใช้งาน
โมดูลการตั้งค่า

DELETE

/positions/delete/{id}

ลบตำแหน่งออกจากระบบ

Parameters

Try it out

Name	Description
id * required	รหัสตำแหน่ง
string (path)	id

Responses

Code	Description	Links
200	✓ ลบตำแหน่งสำเร็จ	No links
404	✗ ไม่พบตำแหน่ง	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

GET

/positions/detail/{id}

ดึงข้อมูลตำแหน่งตาม ID

Parameters

Try it out

Name	Description
id * required	รหัสตำแหน่ง
string (path)	id

Responses

Code	Description	Links
200	✓ ดึงข้อมูลตำแหน่งสำเร็จ	No links
404	✗ ไม่พบตำแหน่ง	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

Products

จัดการข้อมูลสินค้าในร้าน

POST

/products/create/{storeId}

เพิ่มสินค้าใหม่ในร้าน

Parameters

Try it out

Name	Description
storeId * required	รหัสร้านค้า
string (path)	storeId

Request body required

application/json

Example Value | Schema

```
{  "name": "ผลไม้กึ่งสด",  "price": 50,  "stockQty": 100,  "category": "อาหารจานเล็ก"}
```

Responses

Code	Description	Links
201	✓ เพิ่มสินค้าใหม่สำเร็จ	No links
400	✗ ข้อมูลไม่ถูกต้อง	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

GET

/products/all/{storeId}

ดึงรายการสินค้าทั้งหมดของร้าน

Parameters

Try it out

Name	Description
storeId ★ required	รหัสร้านค้า
string (path)	storeId

Responses

Code	Description	Links
200	✅ ดึงข้อมูลสินค้าสำเร็จ	No links
500	❌ เซิร์ฟเวอร์ผิดพลาด	No links

GET

/products/{id}

ดูรายละเอียดสินค้าตาม ID

Parameters

Try it out

Name	Description
id ★ required	รหัสสินค้า
string (path)	id

Responses

Code	Description	Links
200	✅ ดึงรายละเอียดสินค้าสำเร็จ	No links
404	❌ ไม่พบสินค้า	No links
500	❌ เซิร์ฟเวอร์ผิดพลาด	No links

PUT

/products/{id}

แก้ไขข้อมูลสินค้า

Parameters

Try it out

Name	Description
id ★ required	รหัสสินค้า
string (path)	id

Request body required

application/json

Example Value

Schema

```
{
  "name": "string",
  "price": 0,
  "stockQty": 0,
  "category": "string"
}
```

Responses

Code	Description	Links
200	✅ อัปเดตสินค้าสำเร็จ	No links
404	❌ ไม่พบสินค้า	No links
500	❌ เซิร์ฟเวอร์ผิดพลาด	No links

DELETE

/products/{id} ลบสินค้าออกจากระบบ

Parameters

Try it out

Name	Description
id * required string (path)	รหัสสินค้า id

Responses

Code	Description	Links
200	✓ ลบสินค้าสำเร็จ	No links
404	✗ ไม่พบสินค้า	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

POST

/products/{id}/add-serials เพิ่ม Serial Number ให้สินค้า

Parameters

Try it out

Name	Description
id * required string (path)	รหัสสินค้า id

Request body required

application/json

Example Value | Schema

```
{  "serialNumbers": [    "SN001",    "SN002",    "SN003"  ]}
```

Responses

Code	Description	Links
200	✓ เพิ่ม Serial Number สำเร็จ	No links
404	✗ ไม่พบสินค้า	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

Reports

ดึงข้อมูลสรุปยอดขายและรายงานต่าง ๆ ของร้านค้า

GET

/reports/summary/{storeId} ดึงข้อมูลสรุปยอดขายรวมรายวันของร้าน

Parameters

Try it out

Name	Description
storeId * required string (path)	รหัสร้านค้า storeId

Responses

Code	Description	Links
200	✓ ดึงข้อมูลสรุปยอดขายสำเร็จ	No links
401	✗ ไม่มีสิทธิ์เข้าถึง	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

GET

/reports/sales-chart/{storeId}

ดึงข้อมูลยอดขายแบบกราฟ (รายวัน/รายเดือน)

Parameters

Try it out

Name	Description
storeId ★ required	รหัสร้านค้า
string (path)	storeId

Responses

Code	Description	Links
200	✔ ดึงข้อมูลกราฟยอดขายสำเร็จ	No links
401	✖ ไม่มีสิทธิ์เข้าถึง	No links
500	✖ เซิร์ฟเวอร์ผิดพลาด	No links

GET

/reports/top-products/{storeId}

ดึงข้อมูลสินค้าขายดี

Parameters

Try it out

Name	Description
storeId ★ required	รหัสร้านค้า
string (path)	storeId

Responses

Code	Description	Links
200	✔ ดึงข้อมูลสินค้าขายดีสำเร็จ	No links
401	✖ ไม่มีสิทธิ์เข้าถึง	No links
500	✖ เซิร์ฟเวอร์ผิดพลาด	No links

GET

/reports/recent/{storeId}

ดึงรายการออเดอร์ล่าสุดของร้าน

Parameters

Try it out

Name	Description
storeId ★ required	รหัสร้านค้า
string (path)	storeId

Responses

Code	Description	Links
200	✔ ดึงรายการออเดอร์ล่าสุดสำเร็จ	No links
401	✖ ไม่มีสิทธิ์เข้าถึง	No links
500	✖ เซิร์ฟเวอร์ผิดพลาด	No links

Settings

การตั้งค่าร้านค้า (Store Settings)

PUT /settings/{storeId}/name

เปลี่ยนชื่อร้าน

Try it out

Parameters

Name	Description
storeId * required string (path)	รหัสร้านค้า storeId

Request body required

application/json

Example Value | Schema

```
{
  "storeName": "ร้านน้องพลอย POS"
}
```

Responses

Code	Description	Links
200	✓ เปลี่ยนชื่อร้านสำเร็จ	No links
403	✗ ไม่มีสิทธิ์แก้ไขชื่อร้าน	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

PUT /settings/{storeId}/payment

อัปเดตการตั้งค่าการชำระเงิน

Try it out

Parameters

Name	Description
storeId * required string (path)	รหัสร้านค้า storeId

Request body required

application/json

Example Value | Schema

```
{
  "paymentSettings": {
    "cash": true,
    "promptpay": true,
    "promptpayNumber": "0812345678"
  }
}
```

Responses

Code	Description	Links
200	✓ อัปเดตการตั้งค่าชำระเงินสำเร็จ	No links
403	✗ ไม่มีสิทธิ์อัปเดตการชำระเงิน	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

PUT

/settings/{storeId}/logo

อัปโหลดโลโกร้านใหม่



Parameters

Try it out

Name	Description
storeId * required	รหัสร้านค้า
string (path)	storeId
logo * required	ไฟล์รูปโลโกร้าน (.jpg, .png)
(formData)	logo

Responses

Code	Description	Links
200	 อัปโหลดโลโกร้านสำเร็จ	No links
400	 ไฟล์ไม่ถูกต้อง	No links
403	 ไม่มีสิทธิ์อัปเดตโลโก	No links
500	 เซิร์ฟเวอร์ผิดพลาด	No links

PUT

/settings/{storeId}/contact

แก้ไขข้อมูลการติดต่อร้าน



Parameters

Try it out

Name	Description
storeId * required	รหัสร้านค้า
string (path)	storeId

Request body

required

application/json

Example Value

Schema

```
{
  "phone": "0812345678",
  "address": "123/4 ถนนสาทร กรุงเทพฯ"
}
```

Responses

Code	Description	Links
200	 อัปเดตข้อมูลการติดต่อสำเร็จ	No links
403	 ไม่มีสิทธิ์อัปเดตข้อมูล	No links
500	 เซิร์ฟเวอร์ผิดพลาด	No links

DELETE

/settings/{storeId} ลบร้านออกจากระบบ

Parameters

Try it out

Name	Description
storeId * required	รหัสร้านค้า
string (path)	storeId

Responses

Code	Description	Links
200	✔ ลบร้านสำเร็จ	No links
403	✖ ไม่มีสิทธิ์ลบร้าน	No links
500	✖ เซิร์ฟเวอร์ผิดพลาด	No links

Stores

การจัดการร้านค้า (Store Management)

POST

/stores/create-store สร้างร้านใหม่

Parameters

Try it out

No parameters

Request body required

application/json

Example Value | Schema

```
{
  "storeName": "ร้านน้องพลอย POS",
  "storeType": "restaurant",
  "phone": "0812345678",
  "address": "123 ถนนสุขุมวิท กรุงเทพฯ"
}
```

Responses

Code	Description	Links
201	✔ ร้านถูกสร้างเรียบร้อยแล้ว	No links
400	✖ ข้อมูลไม่ถูกต้อง	No links
500	✖ เซิร์ฟเวอร์ผิดพลาด	No links

GET

/stores/my-stores ดึงข้อมูลร้านค้าทั้งหมดของผู้ใช้งาน

Parameters

Try it out

No parameters

Responses

Code	Description	Links
200	✔ ดึงข้อมูลร้านค้าสำเร็จ	No links
401	✖ ไม่พบสิทธิ์เข้าถึง	No links
500	✖ เซิร์ฟเวอร์ผิดพลาด	No links

GET

/stores/{id} ดึงรายละเอียดร้านค้าตาม ID



Parameters

Try it out

Name	Description
id * required string (path)	รหัสร้านค้า id

Responses

Code	Description	Links
200	 ดึงข้อมูลร้านค้าสำเร็จ	No links
404	 ไม่พบร้านค้า	No links
500	 เซิร์ฟเวอร์ผิดพลาด	No links

PUT

/stores/update/{id}

อัปเดตรายละเอียดร้านค้า

Parameters

Try it out

Name	Description
id * required string (path)	รหัสร้านค้า id

Request body required

application/json

▼

Example Value | Schema

```
{
  "storeName": "ร้านน้องพลอยฮิลเดค",
  "phone": "0899999999",
  "address": "456 ถนนพระราม 9 กรุงเทพฯ"
}
```

Responses

Code	Description	Links
200	✓ อัปเดตร้านค้าสำเร็จ	No links
404	✗ ไม่พบร้านค้า	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

Upload

การอัปโหลดไฟล์และรูปภาพ

POST

/upload

อัปโหลดรูปภาพหรือไฟล์ (รองรับ JPG, PNG, GIF, WEBP, PDF)

Parameters

Try it out

No parameters

Request body required

multipart/form-data

▼

file
string(\$binary)

ไฟล์ที่ต้องการอัปโหลด

Responses

Code	Description	Links
200	✓ อัปโหลดไฟล์สำเร็จ Media type application/json ▼ Controls Accept header Example Value Schema <pre>{ "message": "Upload successful", "filePath": "/uploads/file-123456789.jpg" }</pre>	No links
400	✗ รูปแบบไฟล์ไม่ถูกต้องหรือขนาดเกิน 10MB	No links
401	✗ ไม่มีสิทธิ์ (ต้องเข้าสู่ระบบก่อน)	No links
500	✗ เซิร์ฟเวอร์ผิดพลาด	No links

3.5. Report API Checklist

Admin จัดการข้อมูลร้านค้าสำหรับผู้ดูแลระบบ			^
GET	/admin/stores	ดึงข้อมูลร้านค้าทั้งหมด	🔒 ↓
DELETE	/admin/stores/{id}	ลบร้านค้าออกจากระบบ	🔒 ↓
Auth ระบบจัดการการเข้าสู่ระบบและโปรไฟล์ผู้ใช้			^
POST	/auth/register	สมัครสมาชิกใหม่	↓
POST	/auth/login	เข้าสู่ระบบ	↓
GET	/auth/profile	ดึงข้อมูลโปรไฟล์ผู้ใช้ปัจจุบัน	🔒 ↓
Employees จัดการข้อมูลพนักงานในร้านค้า			^
GET	/employees/{storeId}	ดึงข้อมูลพนักงานทั้งหมดในร้าน	🔒 ↓
POST	/employees/create/{storeId}	เพิ่มพนักงานใหม่ในร้าน	🔒 ↓
PUT	/employees/update/{id}	แก้ไขข้อมูลพนักงาน	🔒 ↓
DELETE	/employees/delete/{id}	ลบพนักงานออกจากระบบ	🔒 ↓

4. ผลการดำเนินงาน

โครงการ StoreLink ระบบบริหารจัดการร้านค้า ได้รับการพัฒนาและดำเนินการตามแผนงานที่กำหนดไว้ โดยมีการออกแบบ พัฒนา และทดสอบระบบในแต่ละส่วนจนสามารถทำงานได้อย่างครบถ้วนตามวัตถุประสงค์ที่วางไว้ ผลการดำเนินงานสามารถสรุปได้ดังนี้

1. การออกแบบและพัฒนาระบบ

- ได้ออกแบบระบบตามหลักการของ **Client–Server Architecture** โดยแบ่งออกเป็น 3 ส่วนหลัก ได้แก่
 - Frontend (React.js + Tailwind CSS)** สำหรับสร้างหน้าจอผู้ใช้ที่ตอบสนองรวดเร็วและใช้งานง่าย
 - Backend (Node.js + Express.js)** สำหรับจัดการตรรกะการทำงาน การตรวจสอบสิทธิ์ และการเชื่อมต่อฐานข้อมูล

- **Database (MongoDB)** สำหรับจัดเก็บข้อมูลร้านค้า สินค้า พนักงาน และยอดขาย โดยมีการแยกข้อมูลตามรหัสร้าน (Store ID) อย่างเป็นระบบ
- ได้พัฒนา API ที่รองรับการทำงานครบถ้วน เช่น ระบบล็อกอินด้วย JWT, ระบบจัดการร้านค้า, ระบบสินค้า, ระบบขาย (POS), ระบบพนักงาน และระบบรายงานยอดขาย

2. การทำงานของระบบ (System Functionality)

ระบบสามารถทำงานได้ตรงตามวัตถุประสงค์หลัก ดังนี้

- **ระบบจัดการร้านค้า (Store Management)**
สามารถสร้าง แก้ไข หรือลบร้านค้าได้หลายแห่งภายใต้บัญชีเดียว พร้อมแยกข้อมูลแต่ละร้านอย่างชัดเจน
- **ระบบจัดการสินค้า (Product Management)**
เพิ่ม แก้ไข และลบสินค้าได้ รวมถึงอัปเดตสต็อกสินค้าแบบเรียลไทม์ พร้อมรองรับสินค้าหลากหลายประเภท
- **ระบบขายหน้าร้าน (Point of Sale)**
รองรับการขายสินค้าและบริการ คำนวณยอดรวม ส่วนลด และภาษีโดยอัตโนมัติ รวมถึงออกใบเสร็จได้
- **ระบบจัดการพนักงาน (Employee Management)**
สามารถเพิ่มพนักงาน กำหนดตำแหน่งงาน และสิทธิ์การเข้าถึง (Role Management) เพื่อจำกัดการเข้าถึงข้อมูล
- **ระบบรายงานยอดขาย (Sales Report)**
แสดงยอดขายรายวัน รายเดือน และรายการสินค้าขายดีในรูปแบบกราฟและตาราง เพื่อช่วยในการตัดสินใจทางธุรกิจ
- **ระบบความปลอดภัย (Security)**
ใช้ JWT Authentication ในการยืนยันตัวตนผู้ใช้งาน และมีการเข้ารหัสข้อมูลก่อนจัดเก็บในฐานข้อมูล

3. การทดสอบระบบ (Testing)

- ได้ทำการทดสอบระบบทั้งในส่วน **Unit Test** และ **Integration Test** เพื่อให้มั่นใจว่าการเชื่อมต่อระหว่าง Frontend และ Backend ทำงานถูกต้อง
- ทดสอบการทำงานจริงผ่านเครื่องมือ **Postman** สำหรับ API และ **Browser Testing** สำหรับการใช้งานจริง
- ผลการทดสอบพบว่าระบบสามารถทำงานได้ถูกต้องตามที่ออกแบบ เช่น
 - การเข้าสู่ระบบและตรวจสอบสิทธิ์ผู้ใช้ทำงานถูกต้อง
 - การขายสินค้าผ่านหน้าจอ POS สามารถบันทึกและอัปเดตสต็อกได้ทันที
 - ระบบรายงานสามารถสรุปยอดขายได้ตรงกับข้อมูลจริงในฐานข้อมูล

4. ผลลัพธ์จากการใช้งาน (System Outcomes)

- ผู้ใช้งาน (เจ้าของร้าน) สามารถบริหารร้านค้าได้ครบวงจรในระบบเดียว
- พนักงานสามารถขายสินค้าและตรวจสอบข้อมูลได้ตามสิทธิ์ที่ได้รับ
- ข้อมูลการขายและสต็อกอัปเดตแบบเรียลไทม์ ลดข้อผิดพลาดจากการบันทึกด้วยมือ
- เจ้าของร้านสามารถดูยอดขายผ่านหน้า Dashboard ได้ทุกที่ทุกเวลา
- ระบบสามารถรองรับการเพิ่มร้านค้าใหม่ได้โดยไม่กระทบต่อร้านเดิม (Scalable Architecture)

5. การนำเสนอและสาธิตระบบ

5. ได้จัดทำ สไลด์นำเสนอ และ เดโมระบบจริง (Prototype) โดยแสดงการทำงานของแต่ละโมดูล เช่น

- 5.1. หน้าจอ Dashboard
- 5.2. หน้าขายสินค้า (POS Interface)
- 5.3. หน้าจัดการพนักงาน
- 5.4. หน้ารายงานยอดขาย

6. สรุปผลโครงการและข้อเสนอแนะ

6.1. สรุปผลโครงการ

โครงการ “StoreLink ระบบบริหารจัดการร้านค้า” ได้รับการพัฒนาเพื่อแก้ไขปัญหาที่เจ้าของธุรกิจร้านค้าขนาดเล็กและขนาดกลางประสบอยู่ในปัจจุบัน เช่น การจัดการข้อมูลที่ซ้ำซ้อน การควบคุมสต็อกที่ไม่เป็นระบบ การติดตามยอดขายที่ล่าช้า และการขาดเครื่องมือช่วยวิเคราะห์ข้อมูลเชิงธุรกิจ ระบบนี้จึงถูกออกแบบมาเพื่อช่วยให้การบริหารร้านค้าทำได้สะดวก ถูกต้อง และมีประสิทธิภาพมากยิ่งขึ้น

ผลการดำเนินโครงการแสดงให้เห็นว่า ระบบสามารถทำงานได้ครบถ้วนตามวัตถุประสงค์ที่ตั้งไว้ โดยมีคุณสมบัติสำคัญดังนี้

1. ระบบบริหารจัดการร้านค้า (Store Management)

สามารถสร้างและจัดการร้านค้าหลายแห่งภายใต้บัญชีเดียว โดยข้อมูลของแต่ละร้านถูกแยกออกจากกันอย่างชัดเจน (Data Isolation)

2. ระบบจัดการสินค้าและสต็อก (Product Management)

รองรับการเพิ่ม แก้ไข และลบข้อมูลสินค้า รวมถึงการอัปเดตจำนวนสต็อกแบบเรียลไทม์ เพื่อให้เจ้าของร้านสามารถตรวจสอบและควบคุมได้ตลอดเวลา

3. ระบบขายหน้าร้าน (POS System)

สามารถบันทึกการขายสินค้าได้อย่างรวดเร็ว มีการคำนวณส่วนลด ภาษี และยอดสุทธิอัตโนมัติ พร้อมรองรับการชำระเงินหลายรูปแบบ เช่น เงินสด และพร้อมเพย์ (PromptPay)

4. ระบบจัดการพนักงานและสิทธิ์การเข้าถึง (Employee & Role Management)

เจ้าของร้านสามารถเพิ่มพนักงาน กำหนดตำแหน่ง และควบคุมสิทธิ์การเข้าถึงข้อมูลได้ตาม

หน้าที่ เพื่อเพิ่มความปลอดภัยและความโปร่งใสในการทำงาน

5. ระบบรายงานและสถิติยอดขาย (Sales Report & Dashboard)

แสดงผลยอดขาย สินค้าขายดี และสถิติรายวัน/รายเดือน ในรูปแบบกราฟและตาราง เพื่อช่วยให้เจ้าของร้านสามารถวิเคราะห์และวางแผนธุรกิจได้อย่างมีประสิทธิภาพ

6. ระบบความปลอดภัยของข้อมูล (Security System)

ใช้เทคโนโลยี JWT Authentication ในการยืนยันตัวตนผู้ใช้งาน และมีการเข้ารหัสข้อมูลก่อนบันทึกในฐานข้อมูล เพื่อป้องกันการเข้าถึงโดยไม่ได้รับอนุญาต

การทดสอบระบบในขั้นตอนสุดท้าย (Testing & Evaluation) พบว่า ระบบสามารถทำงานได้ถูกต้องตามข้อกำหนดไว้ ทั้งในส่วน Frontend (React.js + Tailwind CSS) และ Backend (Node.js + Express.js + MongoDB) โดยผู้ใช้งานสามารถเข้าถึงระบบผ่านเว็บเบราว์เซอร์ได้อย่างเสถียรและตอบสนองรวดเร็ว

6.2. ปัญหาที่พบในโครงการ

ระหว่างการพัฒนาโครงการ **StoreLink ระบบบริหารจัดการร้านค้า** ทีมผู้พัฒนาได้พบปัญหาและอุปสรรคหลายประการ ซึ่งส่งผลกระทบต่อกระบวนการพัฒนาและการทดสอบระบบ โดยสามารถสรุปได้ดังนี้

1. ปัญหาด้านการเชื่อมต่อระหว่าง Frontend และ Backend

ในช่วงแรกพบปัญหาเกี่ยวกับการเรียกใช้งาน API และการส่งข้อมูลระหว่างฝั่งผู้ใช้ (React.js) กับฝั่งเซิร์ฟเวอร์ (Node.js + Express.js) ซึ่งเกิดจากการตั้งค่า CORS และการจัดการ Token Authentication ที่ไม่ถูกต้อง ทำให้ต้องปรับปรุงการตั้งค่าการร้องขอข้อมูล และ Middleware ใหม่ให้สอดคล้องกัน

2. ปัญหาด้านการจัดการฐานข้อมูล (MongoDB)

ในการออกแบบโครงสร้างฐานข้อมูลเริ่มต้น พบปัญหาการซ้ำซ้อนของข้อมูลร้านค้าและสินค้าบางส่วน เนื่องจากไม่ได้กำหนดความสัมพันธ์ของเอกสาร (Document Relationship) ชัดเจน จึงต้องปรับโครงสร้างข้อมูลให้มีการอ้างอิงด้วย **storeId** เพื่อแยกข้อมูลแต่ละร้านให้เป็นอิสระ

3. ปัญหาด้านการแสดงผลข้อมูลแบบเรียลไทม์

การอัปเดตข้อมูลสต็อกและยอดขายทันทีหลังการขายบางครั้งแสดงผลล่าช้า เนื่องจากการ

จัดการ State ใน React.js ยังไม่เหมาะสม ต้องปรับปรุงการใช้ `useEffect` และ `axios` ให้ดึงข้อมูลใหม่หลังการอัปเดตสำเร็จ

4. ปัญหาด้านการออกแบบ UI/UX

ในช่วงแรกหน้าจอการทำงานยังซับซ้อนและไม่เป็นมิตรกับผู้ใช้ ทำให้ต้องปรับปรุงการออกแบบส่วนติดต่อผู้ใช้ (User Interface) ด้วย Tailwind CSS เพื่อให้มีความเรียบง่าย สวยงาม และใช้งานสะดวกยิ่งขึ้น

5. ข้อจำกัดด้านเวลาและการทดสอบระบบ

เนื่องจากระยะเวลาในการพัฒนาโครงการมีจำกัด ทำให้การทดสอบระบบในสภาพแวดล้อมจริงยังไม่ครอบคลุมทุกกรณี โดยเฉพาะการใช้งานพร้อมกันหลายบัญชี (Concurrent Users)

6.3. ข้อเสนอแนะ

เพื่อพัฒนาและต่อยอดระบบ **StoreLink** ให้มีความสมบูรณ์และสามารถนำไปใช้งานได้จริงในระดับธุรกิจขนาดใหญ่ ผู้พัฒนาได้เสนอแนวทางปรับปรุงและพัฒนาต่อไปดังนี้

1. พัฒนาแอปพลิเคชันบนโทรศัพท์มือถือ (Mobile Application)

เพื่อเพิ่มความสะดวกในการใช้งานของเจ้าของร้านและพนักงาน โดยสามารถตรวจสอบยอดขายหรือสต็อกจากมือถือได้ทุกที่ทุกเวลา

2. เพิ่มระบบแจ้งเตือนอัตโนมัติ (Notification System)

เช่น แจ้งเตือนเมื่อสต็อกสินค้าใกล้หมด หรือเมื่อมีการขายสินค้าครบตามเป้าหมายประจำวัน เพื่อช่วยให้เจ้าของร้านสามารถบริหารได้อย่างมีประสิทธิภาพมากขึ้น

3. เชื่อมต่อกับระบบบัญชีและการชำระเงินออนไลน์

เช่น การเชื่อมกับ PromptPay API หรือระบบบัญชีออนไลน์ เพื่อช่วยลดขั้นตอนการทำบัญชี และเพิ่มความสะดวกในการตรวจสอบยอดเงิน

4. ปรับปรุงระบบรายงานยอดขายให้ละเอียดมากขึ้น

โดยเพิ่มฟังก์ชันวิเคราะห์ข้อมูลเชิงลึก (Data Analytics) เช่น กำไรต่อสินค้า การเปรียบเทียบยอดขายรายเดือน และแนวโน้มสินค้าในอนาคต

5. พัฒนาระบบสำรองข้อมูล (Backup & Recovery System)

เพื่อป้องกันการสูญหายของข้อมูลในกรณีที่ระบบเกิดข้อผิดพลาดหรือมีปัญหาทางเทคนิค

6. ทดสอบระบบในสภาพแวดล้อมจริงเพิ่มเติม

เพื่อเก็บข้อมูลจากผู้ใช้งานจริง และนำผลตอบรับไปปรับปรุงการใช้งานให้เหมาะสมกับร้านค้าประเภทต่าง ๆ มากยิ่งขึ้น